

Replication of Pig Optimal Strategy

Jay Styles¹, Tina Sardashti¹, Kyle Russell¹, and Joseph Guntrip¹

¹STOR-i CDT, Lancaster University, Lancaster, United Kingdom

Edited by
(Editor)

Received

Published

DOI

1 Introduction

1.1 The game of Pig

Pig is a two-player jeopardy dice game in which the objective is to be the first player to reach 100 points. On each turn, a player repeatedly rolls a single six-sided die and builds up a temporary turn total. If the player rolls a 1, the turn total is lost, and play passes to the opponent. If the player rolls any other value, that value is added to the turn total, and the player may choose either to roll again or to hold. If the player holds, the turn total is added to their overall score, and the turn passes to the opponent.

The rules of Pig are simple, but the strategic problem is not. At every non-terminal state, the player has only two possible actions: roll or hold. However, the value of each action depends on the current player's score, the opponent's score, and the current turn total. Pig, therefore, provides a compact but non-trivial stochastic decision problem.

A common rule of thumb is to hold once the turn total reaches 20. This heuristic has an expected-score interpretation: when a player does not roll a 1, the average successful roll is 4, while the probability of losing the turn total is 1/6. However, maximising expected score in a single turn is not the same as maximising the probability of eventually winning the game. For example, a player who is far behind may need to take greater risks, while a player who is comfortably ahead may benefit from holding earlier.

The work of Neller and Presser [1] addresses this distinction by computing an optimal state-dependent policy for Pig. Their approach models the game in terms of the probability of winning from each possible state and uses value iteration to decide whether rolling or holding gives the larger probability of eventual victory. The resulting policy is substantially more complex than the "hold at 20" heuristic and contains several unintuitive features, particularly when one or both players are close to the target score.

1.2 Aim of the reproduction

This report aims to reproduce the main computational results reported by Neller and Presser [1]. We follow the structure of the original study by first considering the simplified coin-based game Piglet, which is used to demonstrate value iteration on a small state space, before applying the same computational idea to the full game of Pig. Our reproduction focuses on the following results from the original paper:

- The Piglet value-iteration convergence result shown in Figure 2 of the original paper;
- The optimal roll/hold boundary for Pig shown in Figure 3;
- The cross-section of the policy when the opponent's score is fixed at 30, shown in Figure 4;
- The reachable-state visualisations shown in Figures 5 and 6;

Code is available at https://github.com/jguntrip42/pig_review_repo.

- The win-probability contours shown in Figure 7;
- The reported numerical win probabilities for optimal play against both optimal and “hold at 20” opponents.

The purpose of this reproduction is not only to check whether the same qualitative and numerical results can be obtained, but also to examine how easily the original computational workflow can be reconstructed from the published description. All code and documentation for our implementation are provided in the accompanying GitHub repository.¹

1.3 The 5Rs

In addition to reproducing the main results, we assess the original study and our own implementation using the 5Rs of scientific research (as discussed in [2]): replicability, reproducibility, repeatability, re-usability, and re-runability.

- **Replicability** concerns whether the same scientific findings can be obtained through an independent implementation of the described method.
- **Reproducibility** concerns whether the reported results can be regenerated from the supplied materials, code, and documentation.
- **Repeatability** concerns whether rerunning the same code under the same conditions produces the same outputs.
- **Re-usability** concerns whether the code and documentation are sufficiently clear and modular to be adapted for related problems.
- **Re-runability** concerns whether the computational workflow can be executed directly from the supplied repository.

The original paper gives a clear mathematical formulation of Pig and explains the value-iteration method used to compute the optimal policy. However, many practical implementation details expected in modern computational research, such as complete source code, exact software versions, plotting procedures, convergence thresholds, random seeds, and runtime information, are not fully specified. Our reproduction, therefore, considers both the correctness of the reproduced results and the transparency of the computational process used to obtain them.

2 Methods

2.1 State-space formulation

We represent each state of Pig as (i, j, k) , where i is the current player’s score, j is the opponent’s score, and k is the current turn total. The action space is $A = \{\text{roll}, \text{hold}\}$. If $i + k \geq 100$, the current player can hold and win immediately.

$P_{i,j,k}$ denotes the probability that the current player eventually wins from state (i, j, k) , assuming optimal play. For non-terminal states, the optimal probability of winning is

$$P_{i,j,k} = \max(P_{i,j,k,\text{roll}}, P_{i,j,k,\text{hold}}).$$

If the player holds, the turn total is added to their score, and the opponent becomes the current player. Therefore,

$$P_{i,j,k,\text{hold}} = 1 - P_{j,i+k,0}.$$

If the player rolls, then rolling a 1 loses the turn total and passes play to the opponent, while rolling 2–6 increases the turn total. Hence,

$$P_{i,j,k,\text{roll}} = \frac{1}{6} [(1 - P_{j,i,0}) + P_{i,j,k+2} + P_{i,j,k+3} + P_{i,j,k+4} + P_{i,j,k+5} + P_{i,j,k+6}].$$

The optimal policy is obtained by comparing $P_{i,j,k,\text{roll}}$ and $P_{i,j,k,\text{hold}}$ in every state.

¹https://github.com/jguntrip42/pig_review_repo.git

2.2 Value iteration

The probabilities above are mutually dependent, since states may depend on future states in which the roles of the two players have been reversed. For this reason, the values cannot be obtained by a simple backwards calculation. Instead, the original paper uses value iteration.

The game can be viewed as a Markov Decision Process with state space S and action space A . For each state $s \in S$, value iteration repeatedly updates an estimate $V(s)$ of the value of that state. In this reproduction, $V(s)$ represents the probability of eventually winning from state s .

Algorithm 1 Value Iteration

```

1: For each  $s \in S$ , initialise  $V(s)$  arbitrarily.
2: repeat
3:    $\Delta \leftarrow 0$ 
4:   for each  $s \in S$  do
5:      $v \leftarrow V(s)$ 
6:      $V(s) \leftarrow \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V(s')]$ 
7:      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
8:   end for
9: until  $\Delta < \epsilon$ 

```

Here, $P_{ss'}^a$ is the probability of transitioning from state s to state s' after action a , $R_{ss'}^a$ is the immediate reward, γ is the discount factor, and ϵ is the convergence tolerance. As in the original paper, we take $\gamma = 1$, since the objective is the undiscounted probability of eventually winning. Note that in general the algorithm may not converge for $\gamma = 1$, but it does converge for Pig and Piglet.

The Q -function of the algorithm is given by

$$Q(a, s) = \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V(s')].$$

2.3 Piglet validation

Before applying value iteration to the full game of Pig, the original paper introduces Piglet, a simplified version of the game played with a coin rather than a die. In Piglet, the available actions are flip and hold. A successful flip (tossing heads) increases the turn total by 1, while an unsuccessful flip (tossing tails) ends the turn.

We use Piglet as a validation case for our implementation. For a target score of 2, the original paper reports the exact state values

$$\begin{aligned}
P_{0,0,0} &= \frac{4}{7}, & P_{0,0,1} &= \frac{5}{7}, & P_{0,1,0} &= \frac{2}{5}, \\
P_{0,1,1} &= \frac{3}{5}, & P_{1,0,0} &= \frac{4}{5}, & P_{1,1,0} &= \frac{2}{3}.
\end{aligned}$$

Our implementation is first checked by confirming that value iteration converges to these values.

2.4 Value iteration for Piglet

The states we allow for in our value iteration algorithm for Piglet are given by

$$\begin{aligned}
&(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0), (1, 1, 0), \\
&(1, 0, 1), (1, 1, 1), (0, 0, 2), (0, 1, 2), \text{WIN}.
\end{aligned}$$

The possible actions are hold or flip, which we denote by “H” and “F”, respectively. We reward only transitions which end in the winning state “WIN”, so we set $R_{ss'}^a = 0$ whenever $s' \neq \text{WIN}$ and $R_{ss'}^a = 1$ for whenever $s' = \text{WIN}$. We let T denote the target score (so for our simplified version of Piglet, we have $T = 2$). The probability function we choose to be

$$P_{ss'}^a = \begin{cases} \frac{1}{2}, & a = F, s'_1 = s_1, s'_2 = s_2, s'_3 = 0, \\ \frac{1}{2}, & a = F, s'_1 = s_1, s'_2 = s_2, s'_3 = s_3 + 1, \\ 1, & a = H, s'_1 = s_1 + s_3, s'_2 = s_2, s'_3 = 0, s_1 + s_3 \neq T, \\ 1, & a = H, s_1 + s_3 = T, s' = \text{WIN}, \\ 0, & \text{otherwise.} \end{cases}$$

We use an adapted version of the Q -function for Piglet to exploit the symmetry of the game, which is given by

$$\sum_{s'} P_{ss'}^a [R_{ss'}^a + (V(s')\mathbb{I}_s(s') + (1 - V(\sigma'))(1 - \mathbb{I}_s(s')))],$$

where $\mathbb{I}_s(s')$ is an indicator function which takes the value 1 if the action and state transition means that it remains the player's turn (and σ' is the end state from the perspective of the opponent). More formally, letting $s = (s_1, s_2, s_3)$ and $s' = (s'_1, s'_2, s'_3)$ represent the old and new states, we let

$$\mathbb{I}_s(s') = \begin{cases} 1 & s' = \text{WIN or } (s'_1, s'_2, s'_3) = (s_1, s_2, s_3 + 1) \\ 0 & \text{otherwise,} \end{cases}$$

and we define

$$\sigma' = \begin{cases} (s_2, s_1 + s_3, 0) & \text{if } a = H \text{ and } s_1 + s_3 \neq T \\ (s_2, s_1, 0) & \text{if } a = F \text{ and } s_1 + s_3 \neq T \\ (0, 0, 0) & \text{if } s_1 + s_3 = T. \end{cases}$$

Note that the value σ' takes when $s_1 + s_3 = T$ does not matter, because if this is the case, we will have $\mathbb{I}_s(s') = 1$, so $V(\sigma')$ will not contribute to the value update (but it does require the argument for the code to function properly).

We have assumed that $\gamma = 1$, in which case $V(\sigma')$ is the probability of the opponent winning when it switches to their turn. Since the probabilities of winning from a given state are equal for both the player and the opponent, $1 - V(\sigma')$ is exactly the probability of the opponent not winning (thus, we add it to the updated probability for the player winning).

2.5 Value iteration for Pig

After checking the implementation on Piglet, we applied value iteration to the full dice game Pig. In our implementation, a state is represented by $s = (i, j, k)$, where i is the current player's score, j is the opponent's score, and k is the current player's turn total. The value $V(i, j, k)$ is interpreted as the probability that the current player eventually wins from this state, assuming optimal play by both players. For a target score $T = 100$, our code generates all non-terminal states satisfying

$$0 \leq i < T, \quad 0 \leq j < T, \quad 0 \leq k < T - i.$$

The condition $k < T - i$ means that the current player has not yet reached the target during the current turn. If a queried state satisfies $i + k \geq T$, the value function treats it as a terminal winning state and returns value 1. Similarly, if the opponent has already reached the target, the value function returns value 0. At each state, the player compares

the two available actions (roll and hold). If the player holds, the turn total is added to their score, and the turn passes to the opponent. From the opponent's point of view, the next state is $(j, i + k, 0)$. Therefore, the current player's probability of winning after holding is

$$Q_{\text{hold}}(i, j, k) = 1 - V(j, i + k, 0).$$

If the player rolls, each of the six die outcomes is equally likely. Rolling a 1 ends the turn, loses the accumulated turn total, and passes play to the opponent. From the opponent's point of view, the next state is $(j, i, 0)$, so the current player's value for this outcome is $1 - V(j, i, 0)$. For rolls 2, 3, 4, 5, 6, the turn continues, and the roll is added to the current turn total. So, the value of rolling is

$$Q_{\text{roll}}(i, j, k) = \frac{1}{6} \left[1 - V(j, i, 0) + \sum_{r=2}^6 V(i, j, k + r) \right].$$

The value update used in the implementation is therefore

$$V(i, j, k) = \max \{Q_{\text{roll}}(i, j, k), Q_{\text{hold}}(i, j, k)\}.$$

The policy stores the action which gives the larger value. If rolling gives a larger value, the policy is recorded as "roll"; otherwise, it is recorded as "hold".

All state values are initially set to 0. Value iteration is then performed by repeatedly sweeping through the full state list and updating the values in place. Before the iteration begins, the state list is reversed, so that many states closer to the target score are updated earlier in each sweep. This helps the algorithm use information from states close to the target earlier in the calculation, although it is not the same as the fully staged score-sum method described in the original paper.

Originally, we attempted to implement the methodology highlighted in the paper, using a staged value iteration. We did this by separating the value iteration process by the total score sum $(i + j)$, in which states with larger score sums were calculated first and recursively solved towards smaller score sums. This was motivated by the highlighted reason that the state space can only move to equal or greater score sum states, therefore allowing the computational complexity to be reduced. However, we found that without additional methods to reduce the computational complexity, this method was overshadowed by using the aforementioned method. Although our method does not use the proposed "staged" approach, it achieved similar convergence behaviour and was deemed appropriate to use to reproduce the Pig results.

The convergence tolerance used in the implementation is $\epsilon = 10^{-8}$. During each sweep, the largest absolute change in any state value is recorded as

$$\Delta = \max_s |V_{\text{new}}(s) - V_{\text{old}}(s)|.$$

The iteration stops when $\Delta < \epsilon$. If this condition is not reached within 10000 iterations, the outcome is a warning that value iteration did not converge. Otherwise, the final value table and corresponding roll or hold policy are used as the computed optimal strategy for Pig.

2.6 Reachable states

Reachable states are an important part of analysing the optimal policy of the game of Pig. Players may only be able to access select states due to limits formed from rolling the dice, as well as accessible states from an optimally performing player. These two directions outline the basis for two different rules when replicating the results. Firstly, and the simplest, turn totals are limited to only being 5 above the actual roll/hold boundary, as a turn total of exactly the boundary would result in a hold. This forms a cutoff

for the results and, therefore, also limits the ability to access large overhangs, as this would require rolling impossibly large values. The second rule is outlined by the possible states that an optimally performing player may access, regardless of the opposing player's strategy.

The first rule can be explored using a simple geometric rule, using the turn total value for each (i, j) pair alongside the largest possible dice value. The second rule is explored through the use of simulation, by first constructing an optimal strategy using the optimal policy and then simulating the game of Pig against an opponent who holds and rolls at random and collecting all the accessed states. This approach works as a simplistic way to recover all the available reachable states, with improvements such as using more competitive strategies in conjunction with the random strategy to access the very large (i, j) states in fewer macroreplications.

2.7 Re-implementation

Our implementation follows the value-iteration framework described above. The code is organised around four main tasks: computing the Piglet validation case, computing the full Pig optimal policy, generating reproductions of the figures, and simulating games between selected policies.

3 Results

3.1 Replication of Figure 2: Piglet convergence

Our first reproduction target is Figure 2 of the main paper, which shows the convergence of value iteration for the simplified game Piglet with target score 2. This example is useful because the state space is small and the exact state values are reported in the original paper. Reproducing these values provides an initial check that the value-iteration update has been implemented correctly.

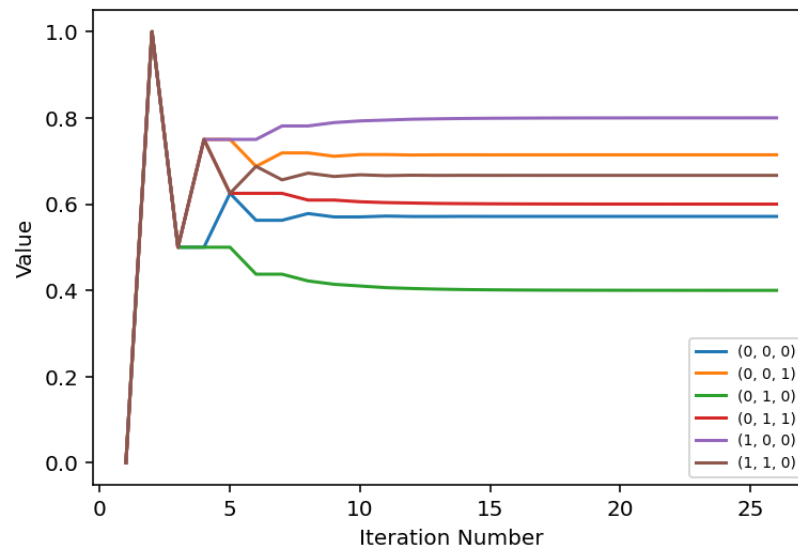


Figure 1. Replication of Figure 2 from Neller and Presser^[1]: convergence of value iteration for Piglet with target score 2.

3.2 Replication of Figure 3: optimal policy boundary

After running value iteration for the full game of Pig, we used the computed state values to extract the optimal policy. At each state, the action with the larger value, roll or hold, was selected as the optimal action. For each score pair (i, j) , we recorded the smallest turn total k at which holding becomes optimal. This gives the roll/hold boundary shown in Figure 2.

States below the boundary surface correspond to rolling, while states on or above the surface correspond to holding. The reproduced boundary shows that the optimal policy is not the same as the simple “hold at 20” rule. Instead, the best action changes depending on the current player’s score and the opponent’s score. The boundary has the same general shape as the original figure in the paper.

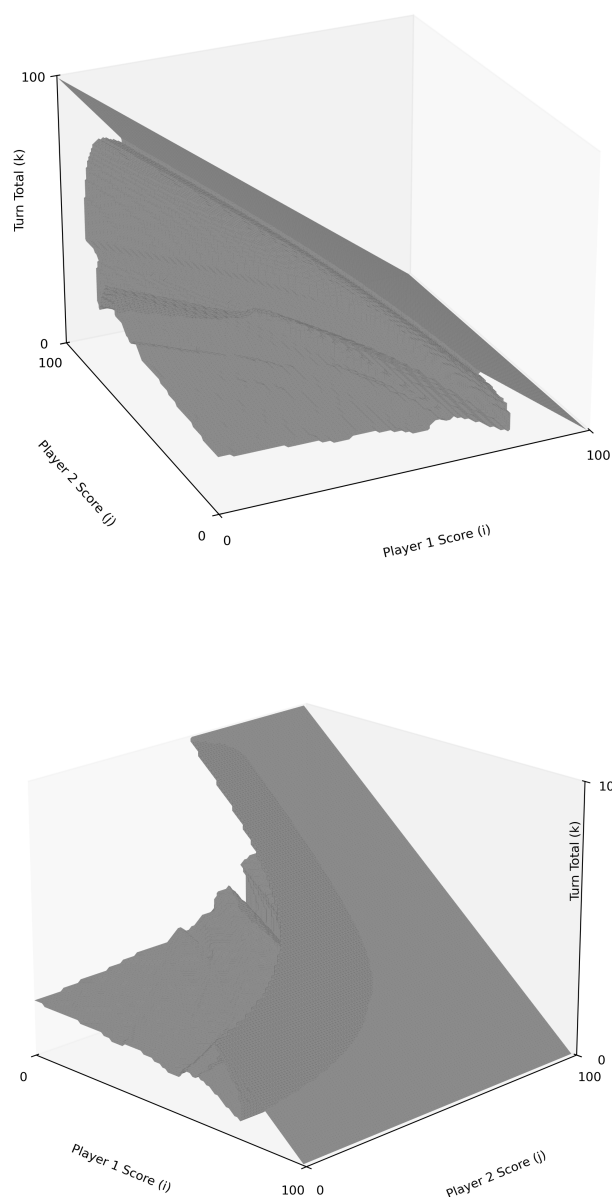


Figure 2. Replication of Figure 3 from [1]: two views of the optimal roll/hold boundary for Pig.

3.3 Replication of Figure 4: cross-section for opponent score 30

Figure 4 of [1] gives a cross-section of the optimal policy when the opponent's score is fixed at 30. This figure is particularly useful because it compares the optimal policy with the “hold at 20” heuristic in a two-dimensional view.

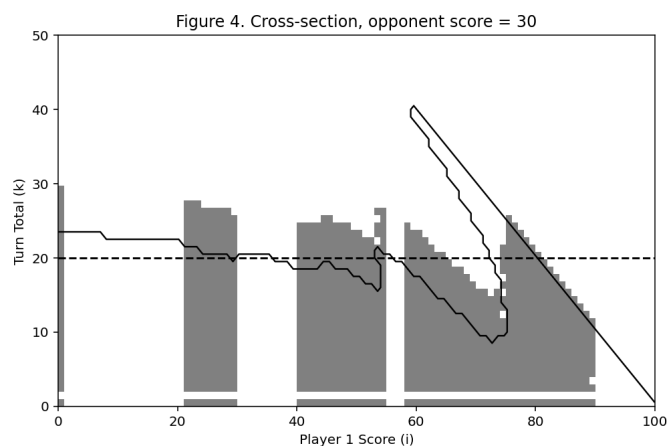


Figure 3. Replication of Figure 4 from [1]: cross-section of the optimal roll/hold boundary when the opponent's score is 30, compared with the "hold at 20" heuristic.

This reproduction allows us to inspect where the optimal policy agrees with the heuristic and where it deviates. In particular, the original figure shows that a player who is behind may need to roll beyond a turn total of 20, while a player with a strong lead may optimally hold before reaching 20.

3.4 Replication of Figure 5 and Figure 6: reachable states

Figure 5 and Figure 6 of Neller and Presser [1] show the states that are reachable by an optimal Pig player. This is different from the full mathematical state space, because some states can only occur if the player has previously made a suboptimal decision or if the player is able to roll more than a 6.

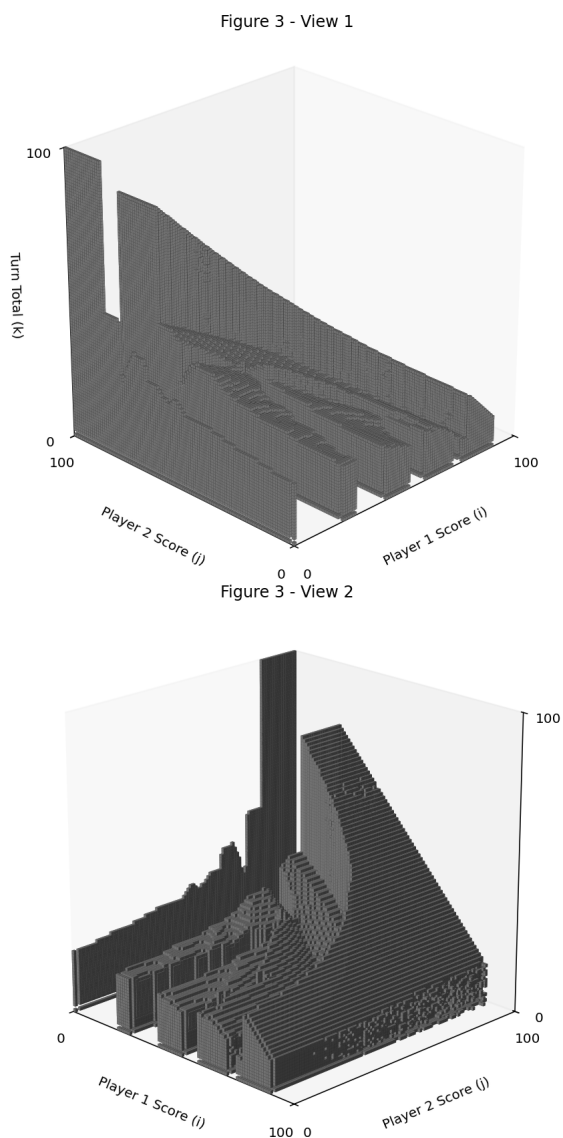


Figure 4. Replications of Figures 5 and 6 from Neller and Presser^[1]: two views of the states reachable by an optimal Pig player.

The reachable-state structure is important because it identifies which parts of the optimal policy are relevant during optimal play. Some visually prominent regions of the full policy surface may correspond to states that an optimal player would never actually reach.

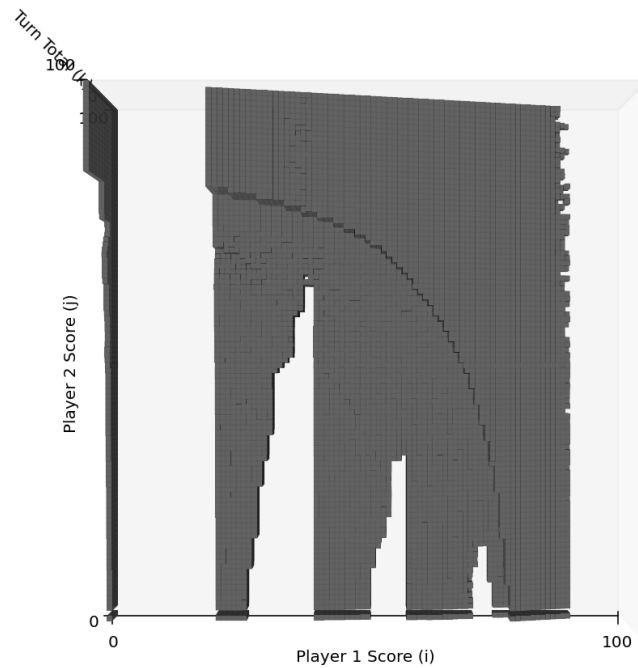


Figure 5. Replication of perspective from Figure 5 from [1], reachable states view from above

3.5 Replication of Figure 7: win-probability contours

Figure 7 of [1] visualises win-probability contours for optimal play. The original paper presents contours corresponding to probabilities 3%, 9%, 27%, and 81%.

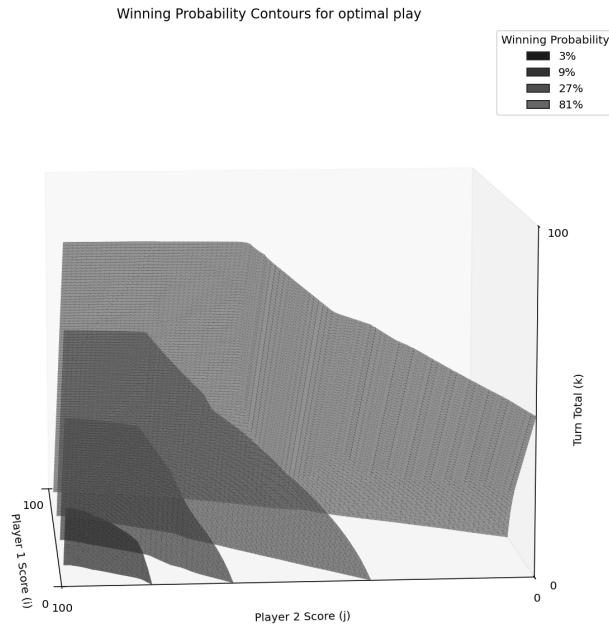


Figure 6. Replication of Figure 7 from [1]: win-probability contours for optimal play.

These contours show how the probability of eventual victory changes across the state space. Reproducing them provides an additional check that the computed value function has the same global structure as the value function reported in the original paper. The winning probability is calculated from the value found in the value iteration algorithm used to solve the pig game. This can then be plotted for a constant value to find states which hold a specific probability of winning.

The recreated plot matches the original paper from [1] with some differences due to the packages used. Figure 6 shows that the probability of winning increases as the player's score increases and the turn total increases. The ridge inwards towards the left of the contours shows that the same winning probability is maintained with a higher player score and lower turn total. This matches the paper.

3.6 Key metrics

In addition to reproducing the figures, we compare selected numerical values reported by Neller and Presser [1]. These metrics test whether the reproduced policy gives similar win probabilities under important matchups.

Table 1. Comparison of win probabilities for the starting player reported by [1] with our reproduced values (we used 10^6 replications in our simulations).

Scenario	Original value	Reproduced value
Optimal vs optimal	0.5306	0.5377
Optimal player starts vs “hold at 20”	0.5874	0.5718
“Hold at 20” player starts vs optimal player	0.4776	0.5004

We also simulated the case where we toss a coin to decide which of the optimal player and the “hold at 20” player starts, and the results showed that the optimal player wins

(approximately) with probability 0.5362 (compared to 0.5549 as stated in the original paper [1]).

4 Discussion

This reproduction aimed to examine whether the main computational findings of Neller and Presser^[1] could be recovered through an independent implementation. The original paper gives a clear mathematical formulation of Pig as a stochastic decision problem and describes how value iteration can be used to compute the optimal roll/hold policy. This made the central methodology possible to follow, especially the use of Piglet as a small validation example before applying the method to the full game of Pig. The most important result is the optimal policy, since the reachable-state plots, contour plots, and policy comparisons all depend on the accuracy of the computed value function.

Our reproduction follows the same overall structure. First, the value-iteration procedure is tested on Piglet, where the exact state values are known. This provides a useful check that the update equations and convergence procedure have been implemented correctly. The method is then extended to the full game of Pig, where the main challenge is the size of the state space and the dependence between states. In the paper, this is handled by exploiting the fact that player scores never decrease, allowing the computation to proceed by descending score sums. Our implementation uses a simpler full-state update order, while keeping the same roll and hold value equations.

4.1 Piglet

In replicating their piglet results, two important conclusions come from our Figure 1. Firstly, the final recorded values and hence their policy are recovered correctly; however, the way in which they converge is slightly different. This can be attributed to asynchronous vs synchronous updates; in this reproduction, synchronous updates of the value within each value were used, as opposed to asynchronous. This is not seen as a significant change, but it does account for the difference in convergence between the Figures. Comparison of exact Piglet state values reported by Neller and Presser^[1] were the same as those found in the replication study. There was some difficulty obtaining the results since the probability and reward functions, as well as the Q -function (required for the value iteration algorithm) were not stated mathematically in the paper.

4.2 Pig

For the full game of Pig, our implementation recovered the main qualitative result of the original paper: the optimal policy is state-dependent and cannot be reduced to the simple “hold at 20” rule. The roll/hold boundary changes with both the current player’s score and the opponent’s score, showing that the best decision depends on the full game state. However, our implementation does not exactly reproduce the staged value-iteration method described by Neller and Presser^[1]. In the paper, states are grouped by score sum and solved in descending order. In our implementation, the full state list is reversed and then updated repeatedly until convergence. This uses the same roll and hold equations, but it is a simpler update scheme. As a result, the main policy structure can still be recovered, but the exact reproduction of the paper’s computational procedure is limited.

4.3 Comparison of Plots

A central part of the reproduction is the optimal policy boundary. This is the most important result because the later visualisations and numerical comparisons all depend on the accuracy of the computed value function. If the reproduced boundary has the

same broad structure as Figure 3 of the original paper, this gives evidence that the main computational result has been recovered. The cross-section with opponent score fixed at 30 provides a more detailed comparison, since it shows directly how optimal play differs from the “hold at 20” heuristic in different regions of the state space.

The reachable-state plots, namely Figures 4 and 5, are also important because they distinguish between states that are mathematically possible and states that can actually occur under optimal play. This matters because some parts of the full optimal policy boundary may correspond to states that would only be reached after earlier suboptimal decisions. Reproducing the reachable-state visualisations therefore provides a further check on both the policy extraction and the interpretation of the original results, though reachable states do not aid in finding the initial solution.

The overall form in Figure 5 is the same as the paper’s version, with the unreachable states removed. Differences in the reachable-state plots between the reproduced and original come predominantly from the ambiguity in the paper. The paper vaguely explains in “The Solution” scenarios that lead to unreachable states without any quantification of this; therefore, to reproduce the results, the simulation approach is developed to find all possible reachable states. This approach relies upon a random strategy, and without large macroreplications, it forms unclear boundaries between unreachable and reachable regions.

4.4 Comparison of Results

There are several possible sources of discrepancy between our results and those in the original paper. Firstly, small differences in convergence tolerance can affect state values close to the roll/hold boundary, where the two actions may have very similar expected values. Second, any results estimated by simulation will depend on the number of realisations and the random seed. Third, the exact plotting procedure used in the original paper is not fully specified, so some visual differences may arise even when the underlying policy is similar. This is particularly relevant for three-dimensional surfaces and the reachable-state plots, where the final appearance depends on plotting choices as well as on the computed values.

Another source of discrepancy is the update order used for the full Pig value iteration. Neller and Presser^[1] describes a staged method, where states are grouped by score sum and solved in descending order. Our Pig implementation uses the same roll and hold equations, but updates the full reversed state list until convergence. This is simpler to implement, but it is not an exact reproduction of the staged computation described in the paper. The paper gives the mathematical method clearly, but it does not provide enough implementation detail, such as the exact update order, data structures, convergence settings, or plotting code, to reproduce the computational workflow exactly. This limits exact reproducibility, even though the main qualitative policy boundary can still be recovered.

4.5 Assessment using the 5Rs

To assess the general ease of reproducibility, the “5Rs” are considered: replicability, reproducibility, repeatability, re-usability and re-runability. These principles are explored by Benureau and Rougier [2] and help to assess the usability, reliability and transparency of research.

Replicability – The original paper is broadly replicable because it provides the key mathematical formulation, including the state representation, the roll and hold equations, and the value-iteration method. These details are sufficient for an independent implementation of the main idea. However, exact replication is harder because the original

paper does not provide a complete repository containing all code used to generate the figures and numerical results.

Reproducibility – The original paper is generally clear, but computational reproducibility is limited by missing practical details such as package versions, plotting scripts, convergence tolerance, runtime, and random seeds for simulation. These omissions do not prevent a new implementation, but they make exact reproduction harder. Explicit mathematical descriptions of the exact value iteration algorithms for the Piglet and Pig would have made the results a lot easier to reproduce. Also, the number of replications for the key metrics obtained from the simulations of different pig strategies against each other would have been helpful for ascertaining why there might be discrepancy from the stated values in the original paper.

Our work aims to improve reproducibility by providing code, documentation, and instructions that link the figures and tables in this report to specific scripts and variable values in the repository.

Repeatability – For deterministic value iteration, repeatability should be high once the implementation, convergence tolerance, and numerical environment are fixed. For simulation results, repeatability requires fixed random seeds and a fixed number of simulations.

Our implementation improves upon the paper’s repeatability by using set seeds and clearly highlighting the used variables in either this report or the repository.

Re-usability – The original paper gives reusable mathematical ideas, but limited reusable software. A reader can adapt the method to related dice games, but they must implement the method themselves.

Our implementation is intended to be re-usable by separating state generation, value iteration, policy extraction, simulation, and plotting into clear components. This should make it possible to modify the target score, test alternative heuristics, or explore related Pig variants. Moreover, this could be improved upon to increase the ability to alter the game itself, the value iteration method or the visualisations through the use of general classes; however, forming a reusable script for these extensions did not form the basis of this report.

Re-runability – Re-runability is the weakest aspect of the original paper. Although the mathematical method is described, there is no complete computational workflow that can be run from start to finish to regenerate the original figures and tables.

Our repository is designed to be re-runnable by providing the code and files needed to regenerate the main outputs. We did this by using a set seed to ensure repeated runs give the same output.

Overall, the reproduction provides a way to assess both the correctness of the original findings and the practical difficulty of regenerating them. The original paper provides sufficient methodological detail to reconstruct the main approach to the value iteration algorithm and figure reproduction, but exact reproduction requires additional choices regarding implementation, convergence, simulation, and visualisation. These choices should be documented carefully, since they can affect both the numerical comparisons and the appearance of the reproduced figures, leading to minor discrepancies and additional unnecessary complexity.

5 Conclusion

The work of Neller and Presser [1] shows that Pig has a substantially more complex optimal strategy than the simple “hold at 20” heuristic suggests. By modelling the game as a

stochastic decision problem and applying value iteration, the original paper computes a state-dependent roll/hold policy based on the probability of eventually winning. This reproduction follows the same approach, using Piglet as a validation example before applying value iteration to the full game of Pig. The reproduced figures and numerical comparisons are used to assess how closely the original results can be recovered. For the Pig policy, our reproduction recovered the main qualitative result that the optimal strategy is state-dependent and cannot be reduced to a fixed “hold at 20” rule. A key takeaway from this attempt to reproduce the findings from their paper is that additional difficulty was added due to the omission of a repository, resulting in us having to infer certain methodological choices. Additionally, the lack of clarity regarding their solutions and methodological choices. Overall, we were able to recover the major conclusions and results from the paper with only minor discrepancies, largely due to visualisation methodology and variable selection.

A Appendix

A.1 Definitions of Algorithm Components

Table 2. Notation used in the value-iteration formulation.

Notation	Meaning
S	Set of all possible states.
$s \in S$	A specific state.
s'	A successor state.
A	Action space. For Pig, $A = \{\text{roll}, \text{hold}\}$.
$V(s)$	Estimated value of state s .
Δ	Maximum change in state value during an iteration.
ϵ	Convergence threshold.
γ	Discount factor.
$P_{ss'}^a$	Probability of transitioning from s to s' after action a .
$R_{ss'}^a$	Immediate reward for the transition from s to s' after action a .

References

1. T. W. Neller and C. G. Presser. “Optimal play of the dice game Pig.” In: **The UMAP Journal** 25.1 (2004).
2. F. C. Benureau and N. P. Rougier. “Re-run, repeat, reproduce, reuse, replicate: transforming code into scientific contributions.” In: **Frontiers in neuroinformatics** 11 (2018), p. 69.